

Projektbericht IoT Gruppe2

STUDIENARBEIT

des Studienganges *Wirtschaftsinformatik Data Science & Consulting*

an der Hochschule für Wirtschaft und Gesellschaft Ludwigshafen

von

Adam Heier, 640697
Leon Schulten, 640514
Nelly Tuedem Ouafo, 637155
Olison Sturm, 640657

08.06.2026

Bearbeitungszeitraum
Dozent der Hochschule

30.04. – 08.06.2026
Prof Dr. Frank Thomé

Inhaltsverzeichnis

1	Einleitung und Projektauftrag	1
1.1	Zielsetzung und Aufgabenstellung	1
1.2	Projektorganisation	1
1.3	Vorgehensweise	2
2	Theoretische Grundlagen	3
2.1	IoT-Grundbegriffe und Systemarchitekturen	3
2.2	Bluetooth / BLE als Nahbereichskommunikation	4
2.3	MQTT: Das zentrale IoT-Protokoll.....	5
2.4	Raspberry Pi als Edge- und Fog-Knoten	6
2.5	Node-RED und Stream Processing.....	6
3	Systemarchitektur und Implementierung	8
3.1	Einordnung in die Gesamtarchitektur	8
3.2	Teilarchitektur Gruppe 2	9
3.3	Bluetooth-Verbindung: TI Sensor zum Raspberry Pi	9
3.3.2	Konfiguration und GATT-Struktur	10
3.4	MQTT-Konfiguration	13
3.5	Node-RED Flows und Filterlogik	16
4	Kritische Würdigung der Projektergebnisse	19
4.1	Zielerreichung	19
4.2	Herausforderungen und Lösungsansätze	19
4.3	Bewertung der gewählten Technologien	19
Anhang A:	Python Code	II

1 Einleitung und Projektauftrag

1.1 Zielsetzung und Aufgabenstellung

Im Rahmen des Gesamtprojekts wurde Gruppe 2 mit folgender Aufgabe betraut: „**TI Sensor Bluetooth- und Message Queuing Telemetry Transport (MQTT) -Anbindung, Edge Computing mit Filterung der Sensordaten für die Zustandsüberwachung**“

[Teilziel 1] Bluetooth-Kommunikation: Aufbau einer stabilen drahtlosen Verbindung zwischen dem TI-Sensortag (Texas Instruments CC2650) und einem Raspberry Pi. Die Bluetooth-Verbindung bildet die erste Schicht der Datenkommunikation und überträgt Rohmesswerte der physischen Sensoren an die Verarbeitungseinheit.

[Teilziel 2] MQTT -Anbindung: Weiterleitung der empfangenen Sensordaten vom Raspberry Pi an den zentralen MQTT-Broker des Gesamtsystems. Dabei sind geeignete Topic-Strukturen zu definieren, ein konsistentes Nachrichtenformat (JSON-Payload) festzulegen sowie Quality-of-Service-Level zu konfigurieren.

[Teilziel 3] Edge Computing mit Node-RED: Aufbau von Verarbeitungsflüssen in Node-RED auf dem Raspberry Pi, die eingehende Sensordaten filtern, validieren und für die Zustandsüberwachung aufbereiten. Die gefilterten Daten werden an nachgelagerte Systemkomponenten weitergeleitet.

1.2 Projektorganisation

Gruppe 2 besteht aus Studierenden des Masterstudiengangs Wirtschaftsinformatik. Alle Gruppenmitglieder haben zu gleichen Anteilen an der praktischen Aufgabenstellung sowie an der schriftlichen Ausfertigung dieses Projektberichts mitgewirkt.

Für die technische Umsetzung standen folgende Systemressourcen zur Verfügung:

Ressource	Bezeichnung / Zugang
Raspberry Pi (Gruppe 3)	IWILR3-6 (<i>gruppe3</i> / <i>pw_gruppe3</i>)
Raspberry Pi mit Node-RED	IWILR4-10, Port 1880
Gemeinsamer Raspberry Pi (Gr. 1 + 2)	IWILR4-11

Die Gruppe 2 steht innerhalb der Systemarchitektur an einer Schlüsselstelle: Sie verbindet die physische Sensorschicht (TI Sensor Tag) mit der Verarbeitungs- und Verteilungsschicht (MQTT, Node-RED) und stellt damit sicher, dass Messdaten strukturiert und gefiltert in das Gesamtsystem einfließen.

1.3 Vorgehensweise

Die Projektarbeit folgte einem iterativen Vorgehen mit schrittweiser Integration der Systemkomponenten:

1. **Grundlagenarbeit** (März/April 2026): Erarbeitung der theoretischen Grundlagen zu Bluetooth, MQTT und Node-RED im Rahmen der Lehrveranstaltung.
2. **Inbetriebnahme Bluetooth** (April/Mai 2026): Aufbau der BLE-Verbindung zwischen Sensor und Raspberry Pi, Testen der Datenübertragung.
3. **MQTT-Konfiguration** (Mai 2026): Einrichtung des Brokers, Definition der Topic-Struktur, Anbindung der Datenquellen.
4. **Node-RED Filterlogik** (Mai 2026): Entwicklung der Verarbeitungsflows, Testläufe und Abstimmung mit den Schnittstellen der anderen Gruppen.
5. **Integration und Dry Run** (03. Juni 2026): Gesamttest aller Arbeitsgruppen gemeinsam.

Begleitend fanden zwei Projektstatustermine am 07. und 21. Mai 2026 statt, bei denen der Fortschritt gegenüber dem Kurs präsentiert und offene Fragen geklärt wurden.

2 Theoretische Grundlagen

2.1 IoT-Grundbegriffe und Systemarchitekturen

Der Begriff *Internet of Things* (IoT) wurde um 2000 von Kevin Ashton geprägt. Ursprünglich auf RFID-Technologie bezogen, bezeichnet er heute die globale Vernetzung digitaler Objekte. Technisch basiert das IoT auf drei Kommunikationsdimensionen: Mensch-zu-Mensch, Mensch-zu-Maschine und als Kernmerkmal der autonomen Maschine-zu-Maschine-Kommunikation (M2M).

Auf dieser Grundlage werden im Folgenden die für das Teilprojekt zentralen Konzepte erläutert: das Cyber-Physische System, IoT-Schichtenarchitekturen sowie Edge- und Cloud-Computing.

2.1.1 Cyber-Physisches System (CPS)

Das im vorliegenden Projekt realisierte System ist ein **Cyber-Physisches System (CPS)**. Dies stellt einen komplexen Systemverbund aus informationstechnischen Komponenten (Hard- und Software), mechanischen bzw. elektronischen Komponenten (LEGO EV3, Sensoren, Aktoren) und einer Netzwerkinfrastruktur mit Echtzeitkommunikation dar. Der Raspberry Pi übernimmt dabei die Rolle des eingebetteten Systems mit Steuerungs- und Datenverarbeitungsfunktion. Die drahtlose Bluetooth-Verbindung sowie das MQTT-Protokoll stellen die Netzwerkkommunikation sicher.

2.1.2 IoT-Referenzarchitekturen

Zur strukturierten Beschreibung von IoT-Systemen werden in der Literatur verschiedene Schichtenmodelle verwendet. Das **Drei-Schichten-Modell** unterscheidet:

- **Perception Layer:** physische Sensoren und Aktoren (hier: LEGO EV3, TI SensorTag)
- **Network Layer:** Konnektivität und Datentransport (hier: Bluetooth, MQTT-Broker auf dem Raspberry Pi)
- **Application Layer:** Datenverarbeitung und Anwendungslogik (hier: Node-RED, ThingsBoard)

Das **Fünf-Schichten-Modell** ergänzt diese Struktur um einen expliziten *Data Processing Layer* (Filterung, Transformation, Aggregation) sowie einen *Business Layer* (Monitoring, Reporting). Gruppe 2 verantwortet im Gesamtsystem insbesondere den Übergang vom Network Layer zum Data Processing Layer.

2.1.3 Edge und Fog Computing

Klassische Cloud-zentrierte IoT-Architekturen senden alle Rohdaten zunächst in eine zentrale Cloud zur Verarbeitung. **Edge Computing** verlagert Rechenleistung dagegen direkt in die Nähe der Datenquelle. **Fog Computing** bezeichnet eine Zwischenstufe, welche die Verarbeitung auf einem lokalen Gateway-Rechner darstellt, der zwischen Sensor und Cloud liegt.

Der Einsatz des Raspberry Pi als Fog-Knoten bietet für das vorliegende Projekt mehrere Vorteile: Erstens werden Latenzen reduziert, da Filterlogik lokal ausgeführt wird, ohne den Weg in die Cloud. Zweitens sinkt das Datenvolumen, das an nachgelagerte Systeme übertragen wird. Drittens wird die Ausfallsicherheit erhöht, da der Raspberry Pi auch ohne Cloud-Anbindung operieren kann.

2.2 Bluetooth / BLE als Nahbereichskommunikation

Bluetooth Low Energy (BLE) ist eine energiesparende Funktechnik nach IEEE 802.15.1 für die drahtlose Nahbereichskommunikation zwischen batteriebetriebenen Geräten.

Ziel im Teilprojekt ist es die drahtlose Verbindung zwischen dem TI CC2650 SensorTag und dem Raspberry Pi herzustellen, über die der PI die Sensor-Messwerte ausliest.

Technische Eigenschaften:

Merkmal	Ausprägung
Frequenzbereich	2,402 – 2,480 GHz
Übertragungsverfahren	Frequenzsprungverfahren (FHSS), 1.600 Sprünge/s
Reichweite	1–100 m (Class 1: 100 m, Class 2: 10 m, Class 3: 1 m)

Merkmal	Ausprägung
Datenrate	64 kbit/s – 723,2 kbit/s; Enhanced Data Rates > 2 Mbit/s
Verbindungstyp	Synchron (SCO, für Sprache), Asynchron (ACL, für Daten)

Der Texas Instruments CC2650 SensorTag nutzt BLE, eine seit Bluetooth 4.0 standardisierte Variante, die auf minimalen Energieverbrauch bei moderaten Datenraten optimiert ist. Die Kommunikationsarchitektur basiert auf dem **Generic Attribute Profile (GATT)**, das Dienste und Charakteristika hierarchisch organisiert. Ein zentrales Gerät (hier: Raspberry Pi als „Central“) liest Messwerte von einem peripheren Gerät (hier: SensorTag als „Peripheral“) über GATT-Charakteristika aus.

2.2.1 Verbindungsaufbau und Pairing

Der Verbindungsaufbau zwischen Raspberry Pi und Bluetooth-Sensor erfolgt in zwei Phasen. Zunächst wird ein **Pairing** durchgeführt, bei dem Geräteschlüssel ausgetauscht und eine vertrauenswürdige Verbindung etabliert wird. Anschließend kann die Verbindung jederzeit ohne erneuten Pairing-Vorgang hergestellt werden. Auf dem Raspberry Pi unter Raspberry Pi OS kann der Verbindungsaufbau über das Kommandozeilenwerkzeug *bluetoothctl* oder programmatisch über die Python-Bibliothek *bluepy* bzw. *bleak* realisiert werden.

2.3 MQTT: Das zentrale IoT-Protokoll

MQTT ist ein schlankes Publish/Subscribe-Nachrichtenprotokoll für die zuverlässige Datenübertragung im IoT-Umfeld über einen zentralen Broker. Seit 2013 als OASIS-Standard etabliert, zeichnet es sich durch einen minimalen Protokoll-Overhead aus, weshalb es sich ideal für ressourcenbeschränkte Geräte und geringe Bandbreiten eignet. Das Ziel im Teilprojekt ist es, die vom Raspberry Pi erfassten Sensordaten via MQTT an den Broker zu übertragen und sie dort für alle nachgelagerten Gruppen strukturiert bereitzustellen.

2.3.1 Publish/Subscribe-Architektur

MQTT folgt dem Publish/Subscribe-Muster, bei dem Publisher und Subscriber nicht direkt, sondern über einen zentralen Message Broker kommunizieren. Der Publisher sendet

Nachrichten an ein definiertes Topic (z.B. Factory/ConditionMonitoring/AirPressure) der Broker verwaltet alle Abonnements und leitet Nachrichten an die jeweiligen Subscriber weiter.

Nachrichtenfluss im Projekt:

TI SensorTag → (Bluetooth) → Raspberry Pi (Publisher) (MQTT, Port 1883) → Mosquitto MQTT Broker → Node-RED (Subscriber)

Architekturvorteil:

Die Entkopplung von Publisher und Subscriber ermöglicht es, neue Subscriber jederzeit hinzuzufügen, ohne den Publisher anpassen zu müssen. Dies ist ein zentrales Designmerkmal skalierbarer IoT-Architekturen.

2.4 Raspberry Pi als Edge- und Fog-Knoten

Der Raspberry Pi ist ein kompakter Einplatinencomputer (SBC) der Raspberry Pi Foundation, der im IoT-Kontext als Gateway oder Edge-/Fog-Knoten zwischen Sensorknoten und übergeordneten Systemen fungiert.

Im vorliegenden Projekt übernimmt der Raspberry Pi (IWILR3-6) mehrere Funktionen gleichzeitig:

1. **Bluetooth-Gateway:** Empfang der Sensorrohdaten über BLE/Bluetooth und Weiterleitung an den Broker.
2. **MQTT-Broker (Mosquitto):** Verwaltung aller Publish/Subscribe-Verbindungen innerhalb des Teilsystems von Gruppe 2.
3. **Edge-Computing-Plattform:** Ausführung der Node-RED-Instanz zur Filterung und Vorverarbeitung der Daten.

Diese Konsolidierung minimiert Latenzen, reduziert das Datenvolumen und gewährleistet Betrieb auch bei Verbindungsunterbrechungen zur Cloud.

2.5 Node-RED und Stream Processing

Node-RED ist eine flow-basierte, visuelle Entwicklungsumgebung auf Node.js-Basis zur Verschaltung von Datenquellen, Logik und Datensinken.

In unserem Teilprojekt filtert, transformiert und drosselt Node-RED auf dem Raspberry Pi (IWILR4-10, Port 1880) die eingehenden Rohdaten und republiziert sie als strukturierte Observation-Nachrichten.

2.5.1 Konzept des Stream Processing

Node-RED realisiert grundlegende Konzepte aus dem Bereich der Data Stream Management Systems (DSMS):

- **Continuous Queries:** Im Gegensatz zu klassischen Datenbankabfragen verarbeiten Continuous Queries fortlaufend eingehende Datenströme, welche in Node-RED durch Function- und Switch-Nodes umgesetzt sind.
- **Windowing:** Durch Tumbling Windows werden Datenpunkte zeitlich aggregiert (z. B. 60-Sekunden-Durchschnitt), was das Datenvolumen reduziert, ohne relevante Trendaussagen zu verlieren.
- **Filterung und Schwellenwertüberwachung:** Switch-Nodes prüfen eingehende Messwerte gegen definierte Bedingungen und leiten Nachrichten entsprechend in unterschiedliche Verarbeitungspfade (z. B. Lichtintensität nimmt zu oder ab, Temperatur überschreitet 50 °C).

2.5.2 Relevante Node-Typen

Node-Typ	Funktion im Projekt
MQTT Input	Abonniert Topics des Brokers
Function	JavaScript-Logik für Transformation und Validierung
Switch	Conditional Routing (Schwellenwertprüfung)
JSON	Parsen und Serialisieren von JSON-Payloads
MQTT Output	Publiziert gefilterte Daten an neue Topics
Debug	Ausgabe im Node-RED Dashboard zur Fehlersuche

3 Systemarchitektur und Implementierung

3.1 Einordnung in die Gesamtarchitektur

Das Gesamtprojekt „LEGO Mindstorms Color Sorter meets Industrie 4.0“ ist auf mehrere Arbeitsgruppen aufgeteilt, die jeweils Teilaufgaben der vollständigen IoT-Wertschöpfungskette übernehmen. Die folgende Tabelle gibt einen Überblick über die Aufgabenverteilung:

Gruppe	Aufgabe
Gruppe 1	ERP-Einrichtung für Fertigungssteuerung; Web-Services bereitstellen und anbinden; MQTT-Umwandlung In-/Outbound
Gruppe 2	TI Sensor Bluetooth-/MQTT-Anbindung, Edge/Fog Computing, Node-RED Filterung
Gruppe 3	Data Streaming, Datenharmonisierung & - persistierung zur Sensordatenanalyse und Zustandsüberwachung
Gruppe 4	Dashboard-Entwicklung mit MQTT- Anbindung; Regeldefinition & Digital Twin zur Zustandsüberwachung

Gruppe 2 nimmt in dieser Architektur eine Brückenfunktion ein: Sie empfängt Rohmesswerte aus der physischen Welt (Bluetooth-Sensordaten) und stellt sie strukturiert und gefiltert für alle nachgelagerten Gruppen über den MQTT-Broker bereit.

3.2 Teilarchitektur Gruppe 2

Die Systemarchitektur von Gruppe 2 lässt sich in drei Schichten einordnen, die dem IoT-Drei-Schichten-Modell entsprechen:

Perception Layer: Der TI SensorTag CC2650 erfasst physikalische Messwerte (Temperatur, Luftfeuchtigkeit, Lichtintensität, Luftdruck) und sendet diese per BLE.

Network Layer: Der Raspberry Pi (IWILR3-6) fungiert als Gateway. Er empfängt die BLE-Daten, wandelt sie in ein standardisiertes JSON-Format um und publiziert sie über den lokal laufenden Mosquitto MQTT-Broker.

Data Processing Layer: Node-RED (IWILR4-10) abonniert die MQTT-Topics, filtert die Rohdaten nach definierten Regeln und leitet relevante Zustandsmeldungen an die nachgelagerten Systeme weiter.

Die Einordnung in das Fog-Computing-Paradigma macht deutlich, dass Gruppe 2 keine reine Datendurchleitung realisiert, sondern aktiv Datenvorverarbeitung (Aggregation, Filterung, Formatierung) am Edge durchführt. Dies ist ein zentrales Merkmal moderner IoT-Architekturen.

3.3 Bluetooth-Verbindung: TI Sensor zum Raspberry Pi

Für die Bluetooth-Kommunikation und die direkte MQTT-Weiterleitung wurden folgende Komponenten eingesetzt:

- **Sensorknoten:** Texas Instruments CC2650 SensorTag (Bluetooth Low Energy 4.0)
- **Gateway:** Raspberry Pi mit integriertem Bluetooth-Adapter unter Raspberry Pi OS
- **Programmiersprache:** Python 3 (asynchron, *asyncio*)
- **BLE-Bibliothek:** *bleak*, eine plattformunabhängige Python-Bibliothek für BLE-Kommunikation über das GATT-Protokoll
- **MQTT-Bibliothek:** *paho-mqtt* (Eclipse Paho), publiziert die ausgelesenen Messwerte direkt an den Broker

3.3.1 Asynchrone Programmarchitektur

Die Implementierung basiert auf dem asynchronen Python-Framework *asyncio*. Dieses ist notwendig, da der Bluetooth-Listener (BLE-Notifications von *bleak*) und der MQTT-Netzwerk-

Loop (paho-mqtt) gleichzeitig und ohne gegenseitige Blockierung betrieben werden müssen. In einem synchronen Ablaufmodell würde das Warten auf eingehende BLE-Notifications den MQTT-Thread blockieren und umgekehrt. Durch `asyncio` können beide I/O-gebundenen Operationen kooperativ in einer einzigen Event-Loop interleaved werden: Sobald ein BLE-Paket eintrifft, wird der Notification-Handler als Coroutine aufgerufen und gibt die Kontrolle danach wieder an die Loop ab, die in der Zwischenzeit MQTT-Keepalive-Pakete versenden kann.

Das paho-MQTT-Client-Objekt wird mit `loop_start()` in einem separaten Hintergrund-Thread gestartet, sodass der `asyncio`-Event-Loop des Hauptprogramms nicht blockiert wird. Beide Ausführungsstränge kommunizieren ausschließlich über den Thread-sicheren Publish-Aufruf `client.publish()`.

3.3.2 Konfiguration und GATT-Struktur

In der Konfiguration werden die MAC-Adresse des SensorTag, der Broker-Hostname sowie das MQTT-Eingangs-Topic festgelegt. Das SENSORS-Dictionary bildet die drei genutzten BLE-Sensoren auf ihre vollständigen GATT-UUIDs (service, data, config, period) ab. Diese UUIDs sind im offiziellen TI CC2650 SensorTag User's Guide spezifiziert und wurden zusätzlich durch Auslesen der Service- und Charakteristik-Struktur des Geräts mit einem BLE-Scanner verifiziert:

```
# --- KONFIGURATION ---
ADDRESS      = "98:07:2D:27:F1:86"
MQTT_BROKER  = "IWILR4-11.CAMPUS.fh-ludwigshafen.de"
MQTT_TOPIC   = "Sensor"

# UUIDs für den TI CC2650 SensorTag (vollständige 128-Bit-UUIDs)
SENSORS = {
    "humidity": { "data": "f000aa21-0451-4000-b000-000000000000",
                  "config": "f000aa22-0451-4000-b000-000000000000", ... },
    "barometer": { "data": "f000aa41-0451-4000-b000-000000000000",
                  "config": "f000aa42-0451-4000-b000-000000000000", ... },
    "optical_lux": { "data": "f000aa71-0451-4000-b000-000000000000",
                   "config": "f000aa72-0451-4000-b000-000000000000", ... },
}
```

3.3.3 Verbindungsaufbau und Sensoraktivierung (GATT-Protokoll)

Beim Programmstart verbindet sich der Paho-MQTT-Client mit dem Broker (Port 1883) und startet seinen Netzwerk-Loop in einem Hintergrund-Thread. Anschließend scannt das Skript per `BleakScanner.find_device_by_address()` nach dem SensorTag und öffnet eine BleakClient-Verbindung.

Für jeden der drei Sensoren werden zwei GATT-Operationen ausgeführt: Erstens wird der Sensor durch Schreiben des Byte-Werts `0x01` in seine Config-Charakteristik aktiviert. Dies entspricht dem offiziellen Aktivierungsbefehl gemäß TI-Datenblatt (SensorTag User's Guide, Rev. E): Im Auslieferungszustand befinden sich alle Sensoren des CC2650 im Standby-Modus, um Energie zu sparen; erst nach dem Schreibbefehl beginnen sie mit der Messung. Zweitens werden über `start_notify()` BLE-Notifications für die jeweilige Daten-Charakteristik abonniert. Das Skript läuft danach in einer Endlosschleife (`asyncio.sleep(1)`), bis es manuell beendet wird.

async with BleakClient(device) **as** client:

for name, uuids **in** SENSORS.items():

await client.write_gatt_char(uuids["config"], b"\x01") # *Sensor einschalten*

await client.start_notify(uuids["data"], notification_handler)

 print(f"-> {name} aktiviert.")

while True:

await asyncio.sleep(1) # *Event-Loop einschalten*

3.3.1 Dekodierung der Rohdaten und Konvertierung

Bei jeder neuen Messung ruft bleak den `notification_handler` auf. Da der TI CC2650 SensorTag aus Energiespargründen binäre Byte-Ströme statt fertig formatierter Zeichenketten übermittelt, müssen die empfangenen Hexadezimalwerte auf dem Raspberry Pi interpretiert und in physikalische Einheiten konvertiert werden. Der Handler identifiziert den Sensor anhand der sendenden Daten-UUID und wendet die datenblattspezifische Konvertierungsformel an.

Temperatur und Luftfeuchtigkeit (Humidity Sensor)

Der Feuchtesensor übermittelt ein kombiniertes 4-Byte-Paket. Die ersten zwei Bytes repräsentieren den Roh-Temperaturwert (`raw_temp`), die hinteren zwei Bytes den Roh-Feuchtigkeitswert (`raw_hum`), jeweils als vorzeichenloser 16-Bit-Integer (little-endian). Die

Transformation in physikalische Werte erfolgt nach den herstellerspezifischen Algorithmen aus dem TI HDC1000 Sensor-Datenblatt:

$$T [^{\circ}\text{C}] = (\text{raw_temp} / 65536) \times 165 - 40$$

$$H [\%RH] = (\text{raw_hum} / 65536) \times 100$$

Luftdruck (Barometer Sensor)

Der Barometersensor übermittelt den Rohwert als vorzeichenloser 32-Bit-Integer in Pa × 100. Die Konvertierung in hPa erfolgt durch einfache Division:

$$p [\text{hPa}] = \text{raw_pressure} / 100$$

Lichtintensität (Optical Lux Sensor)

Der Helligkeitssensor liefert einen 16-Bit-Rohwert, der intern in ein Gleitkommaformat mit Exponent und Mantisse kodiert ist. Die Berechnung der Lichtintensität in Lux erfolgt anschließend nach der Formel aus dem TI OPT3001 Datenblatt:

$$\text{Lux} = m \times 0,01 \times 2^e$$

Der Faktor 0,01 entspricht der Auflösung des Sensors in der kleinsten Messstufe (Exponent e = 0). Durch die Potenzierung mit 2^e wird der messbereichsabhängige Skalierungsfaktor berücksichtigt, der es dem Sensor erlaubt, sowohl sehr niedrige als auch sehr hohe Beleuchtungsstärken (bis ca. 83.865 Lux) mit hoher Auflösung darzustellen.

Übersicht: BLE-Sensoren und Dekodierung

Bemerkenswert ist, dass der Feuchtesensor in einem Paket zwei Größen liefert – so entstehen aus drei BLE-Sensoren die vier Messgrößen, die später im Node-RED-Splitter getrennt werden (vgl. Abschnitt 3.5):

BLE-Sensor	Rohdaten-Dekodierung	Veröffentlichte Messgröße(n)
<i>humidity</i>	<i>Temp</i> = raw/65536 · 165 – 40; <i>Hum</i> = raw/65536 · 100	<i>Temperature</i> (°C), <i>Humidity</i> (%RH)
<i>barometer</i>	<i>Druck</i> = raw/100	<i>AirPressure</i> (hPa)
<i>optical_lux</i>	<i>Lux</i> = m · 0,01 · 2 ^e (Mantisse m, Exponent e aus 16-Bit-Wert)	<i>Illuminance</i> (lux)

3.3.2 MQTT-Ausgabe: JSON-Format

Durch die Vorverarbeitung direkt auf dem Edge-Knoten (Raspberry Pi) werden nachgelagerte Systeme wie Node-RED vollständig von rechenintensiven Bit-Operationen entlastet und erhalten direkt ein standardisiertes, lesbares JSON-Objekt. Die Funktion `publish()` verpackt jeden konvertierten Messwert in ein einheitliches JSON-Format und sendet es auf dem Topic `Sensor` an den Broker (vgl. Abschnitt 3.4):

```
{  
  "device": "ti_cc2650",  
  "type": "Illuminance",  
  "value": 38.99,  
  "timestamp": 1780051874.037  
}
```

Das Feld `type` entspricht dabei der physikalischen Messgröße (z. B. Temperature, Humidity, AirPressure, Illuminance) und ermöglicht dem nachgelagerten Node-RED-Splitter, die eingehenden Nachrichten ohne zusätzliche Dekodierungslogik direkt den entsprechenden Verarbeitungszweigen zuzuordnen.

3.4 MQTT-Konfiguration

3.4.1 Broker-Setup

Als MQTT-Broker dient der auf dem Raspberry Pi **IWILR4-11** (*IWILR4-11.CAMPUS.fh-ludwigshafen.de*) betriebene Dienst, der für alle Gruppen des Projekts als gemeinsame Kommunikationsinfrastruktur bereitsteht. Die Verbindung erfolgt unverschlüsselt über **Port 1883** (MQTT 3.1.1, *autoConnect: true*). Im Hochschulnetz ist dies ausreichend; für einen Produktivbetrieb wäre Port 8883 mit TLS/SSL zu verwenden.

3.4.2 Topic-Struktur und Datenpipeline

Die MQTT-Kommunikation von Gruppe 2 erfolgt in zwei Stufen. Das Python-Skript *sensor.py* auf dem Raspberry Pi publiziert die Rohdaten des TI SensorTag auf dem Eingangs-Topic `Sensor`. Node-RED abonniert dieses Topic, verarbeitet die Daten und republiziert sie auf

separaten, semantisch strukturierten Ausgangs-Topics unter dem Präfix *Factory/ConditionMonitoring/*:

Stufe	Topic	Inhalt
Eingang (Rohdaten)	<i>Sensor</i>	JSON mit <i>device</i> , <i>type</i> , <i>value</i> , <i>timestamp</i>
Ausgang	<i>Factory/ConditionMonitoring/AirPressure</i>	Luftdruck in hPa (Observation-Format)
Ausgang	<i>Factory/ConditionMonitoring/Humidity</i>	Luftfeuchtigkeit in %RH (Observation-Format)
Ausgang	<i>Factory/ConditionMonitoring/Illuminance</i>	Lichtintensität in lux (Observation-Format)
Ausgang	<i>Factory/ConditionMonitoring/Temperature</i>	Temperatur in °C (Observation-Format)

3.4.3 Rohdaten-Payload (Topic: *Sensor*)

Das Python-Skript sendet für jeden Messwert ein kompaktes JSON-Objekt mit den Feldern *device*, *type*, *value* und *timestamp* (Format vgl. Abschnitt 3.3). Die folgende Abbildung zeigt eingehende Rohdaten in einem MQTT-Client, der das Topic *Sensor* abonniert hat:



MQTT Rohdaten auf Topic Sensor

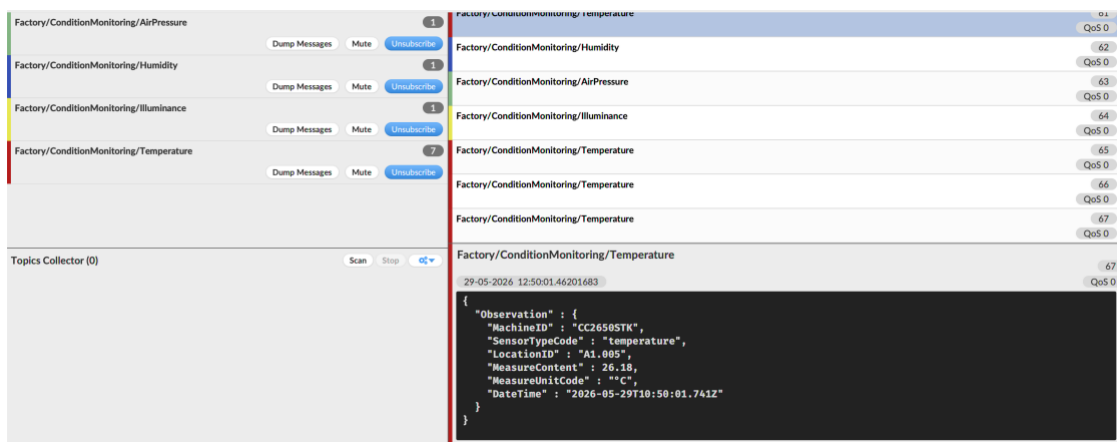
Abbildung 1: MQTT-Rohdaten-Payload auf Topic „Sensor“ (eigene Darstellung, 2026)

3.4.4 Observation-Payload (Ausgangs-Topics)

Node-RED transformiert die Rohdaten in ein standardisiertes **Observation-Format**, das alle für die nachgelagerten Gruppen (Datenbank, ThingsBoard) relevanten Metadaten enthält: Geräte-ID, Sensortyp, Standort (Raum A1.005), Messwert mit Einheit sowie ISO-8601-Zeitstempel.

```
{
  "Observation": {
    "MachineID": "CC2650STK",
    "SensorTypeCode": "Temperature",
    "LocationID": "A1.005",
    "MeasureContent": 26.18,
    "MeasureUnitCode": "°C",
    "DateTime": "2026-05-29T10:50:01.741Z"
  }
}
```

Die folgende Abbildung zeigt den MQTT Explorer mit den vier abonnierten Ausgangs-Topics und einem Beispiel einer eingehenden Temperatur-Observation:



MQTT Broker mit Observation-Nachrichten

Abbildung 2: MQTT-Broker mit eingehenden Observation-Nachrichten (eigene Darstellung, 2026)

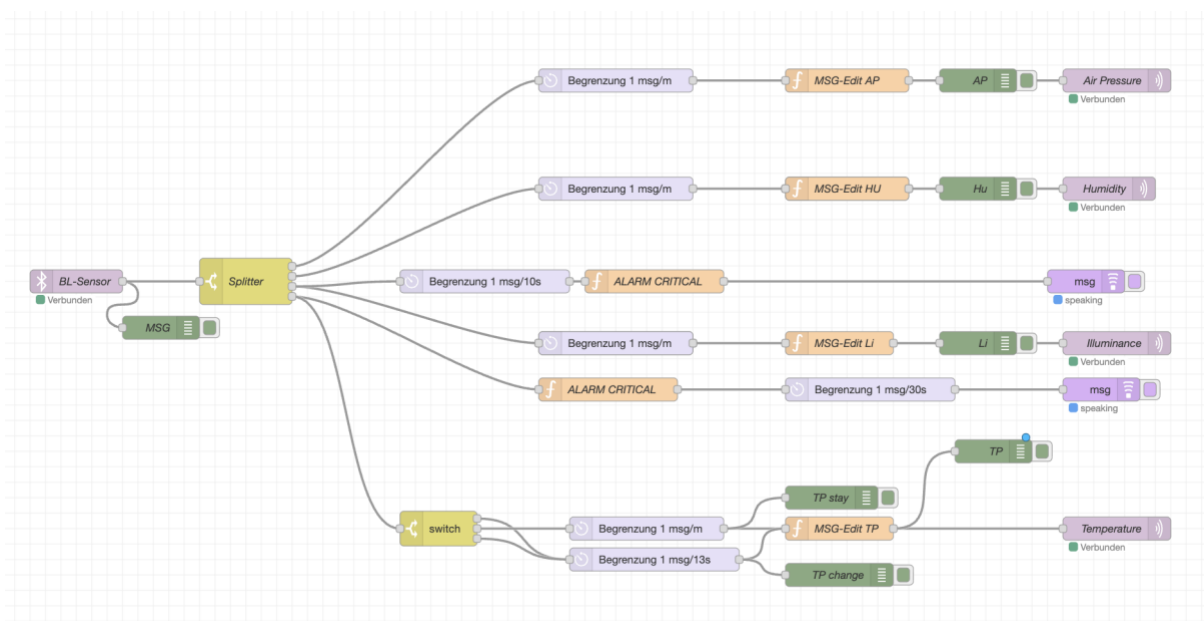
Hinweis: Die Screenshots in Abbildung 1 und 2 entstanden in einer frühen Testphase, in der die Sensortypen noch mit den ursprünglichen Bezeichnungen (*lux*, *temperature* usw.)

gekennzeichnet waren. In der finalen Implementierung tragen sie die mit den Ausgangs-Topics konsistenten Namen (*Illuminance*, *Temperature* usw.).

3.5 Node-RED Flows und Filterlogik

3.5.1 Flow-Übersicht

Die Node-RED-Instanz auf IWILR4-10 (Port 1880) enthält einen gemeinsamen Flow für alle vier Sensortypen des TI CC2650 SensorTag. Die folgende Abbildung zeigt den vollständigen Flow:



Node-RED Flow Gruppe 2

Abbildung 3: Node-RED Flow Gruppe 2 (eigene Darstellung, 2026)

3.5.2 Eingabe: MQTT Input Node „BL-Sensor“

Der Flow startet mit einem **MQTT-Input-Node** namens „BL-Sensor“, der das Topic *Sensor* auf dem Broker *IWILR4-11* mit QoS 2 abonniert. Alle eingehenden Rohdaten des TI SensorTag laufen über diesen einzelnen Eingang. Parallel dazu gibt ein **Debug-Node** „msg“ jede eingehende Nachricht im Node-RED-Debug-Panel aus – nützlich zur Laufzeitprüfung.

3.5.3 Verteilung: Switch-Node „Splitter“

Der **Switch-Node „Splitter“** verzweigt den eingehenden Datenstrom anhand des Feldes *payload.type* in vier Ausgänge. Die Prüfung erfolgt über **JSONata-Ausdrücke**. Eine für Node-RED typische Abfragesprache für JSON-Daten:

Ausgang	JSONata-Bedingung	Sensortyp
1	<i>payload.type</i> = "AirPressure"	Luftdruck
2	<i>payload.type</i> = "Humidity"	Luftfeuchtigkeit
3	<i>payload.type</i> = "Illuminance"	Lichtintensität
4	<i>payload.type</i> = "Temperature"	Temperatur

Da *checkall: true* gesetzt ist, werden alle Regeln geprüft – eine Nachricht kann theoretisch mehrere Ausgänge triggern, was bei disjunkten Typen jedoch nicht eintritt.

3.5.4 Verarbeitungszweige

Jeder der vier Ausgänge des Splitters durchläuft eine eigene Verarbeitungskette aus drei Basiskomponenten:

1. Rate-Limiter (Delay-Node, *drop: true*): Begrenzt den Durchsatz auf **1 Nachricht pro Minute** und verwirft überschüssige Nachrichten. Dies ist die zentrale Datenreduktionsmaßnahme: Da der SensorTag mehrfach pro Sekunde Messwerte sendet, reduziert der Rate-Limiter das Datenvolumen auf das nötige Minimum für die Zustandsüberwachung.

2. Function-Node „MSG-Edit“: Transformiert die Rohdaten in das standardisierte Observation-Format (vgl. Abschnitt 3.4). Der JavaScript-Code ist für alle Sensoren strukturell identisch; lediglich *MeasureUnitCode* und Zeitstempel-Berechnung unterscheiden sich leicht. Beispiel für den Temperatur-Node:

```
msg.payload = {  
  Observation: {  
    MachineID: "CC2650STK",  
    SensorTypeCode: msg.payload.type,  
    LocationID: "A1.005",  
    MeasureContent: msg.payload.value,  
  }  
}
```

```

    MeasureUnitCode: "°C",
    DateTime: new Date(msg.payload.timestamp * 1000).toISOString()
  }
};
return msg;

```

3. MQTT-Output-Node: Publiziert die formatierte Observation auf dem jeweiligen Ausgangs-Topic (*Factory/ConditionMonitoring/Temperature* usw.).

3.5.5 Erweiterte Logik: Alarm und Trendüberwachung

Für Temperatur und Licht sind zusätzliche Verarbeitungspfade implementiert:

[Temperatur] Alarm-Logik: Ein separater **Function-Node** „**ALARM CRITICAL**“ prüft kritische Schwellenwerte:

- Unter 20 °C → Meldung: „*Caution! Temperature is below 20 degrees.*“
- Über 40 °C → Meldung: „*Alarm! Temperature is above 40 degrees!*“

Bei Auslösung wird die Alarmmeldung über einen Delay-Node (max. 1 Alarm alle 30 Sekunden) an einen **Play-Audio-Node** weitergeleitet, der eine Sprachausgabe erzeugt.

[Temperatur] Trendüberwachung: Ein zusätzlicher **Switch-Node** vergleicht jeden Temperaturwert mit dem vorherigen Wert (Referenz: *prev*). Bei drei möglichen Ausgängen (gestiegen / gleich / gesunken) wird differenziert:

- **Wertänderung** (gestiegen oder gesunken): schnellere Weiterleitung über Delay 13 Sekunden → sofortige Aktualisierung des MQTT-Topics
- **Wert unverändert:** gedrosselter Pfad über Delay 1/Minute → reguläres Update

Dieses Muster realisiert ein **ereignisbasiertes Reporting**: Änderungen werden zeitnah gemeldet, Wiederholungen gleicher Werte dagegen stark gedrosselt.

[Illuminance] Alarm-Logik: Analog zur Temperatur prüft ein Function-Node den Lux-Wert:

- Unter 100 lx und über 10 lx → „*Caution! It is getting darker!*“
- Unter 10 lx → „*Alarm! I can't see anything!*“

Der Delay-Node vor dem Play-Audio-Node begrenzt Licht-Alarme auf maximal einen alle 10 Sekunde

4 Kritische Würdigung der Projektergebnisse

4.1 Zielerreichung

Alle drei definierten Teilziele wurden erreicht: Die Bluetooth-Verbindung zwischen TI SensorTag und Raspberry Pi konnte stabil aufgebaut werden, der Mosquitto-Broker verarbeitet sämtliche Publish/Subscribe-Verbindungen zuverlässig mit korrekt bedienten Topics und einheitlichen JSON-Payloads, und die Node-RED-Flows verarbeiten die Nachrichten ohne erkennbaren Rückstau. Filterfunktionen wie Schwellenwertüberwachung, Validierung und Aggregation arbeiten korrekt, und die gefilterten Daten werden erfolgreich an ThingsBoard (Gruppe 4) und die Datenbank (Gruppe 3) weitergeleitet.

4.2 Herausforderungen und Lösungsansätze

Während der Umsetzung traten vor allem Herausforderungen bei der Stabilität der Bluetooth-Verbindung sowie bei der gruppenübergreifenden Koordination der MQTT-Topic-Struktur auf. Letztere erforderte mehrere Iterationen, da nachgelagerte Gruppen (insbesondere Gruppe 3 und 4) spezifische Anforderungen an das Payload-Format hatten.

4.3 Bewertung der gewählten Technologien

Die eingesetzten Technologien erwiesen sich als gut geeignet: Bluetooth deckt mit bis zu 10 m Reichweite (Class 2) die Laborumgebung vollständig ab und ist energiesparend (BLE). Alternativen wie ZigBee, LoRaWAN oder WLAN wären erst bei Outdoor-Szenarien oder größeren Entfernungen relevant. MQTT passt mit minimalem Overhead, QoS-Unterstützung und dem Publish/Subscribe-Mechanismus ideal zu einem System mit einem Produzenten und mehreren Konsumenten. Node-RED bietet schließlich eine ausgezeichnete Balance aus Flexibilität und Einfachheit. Leistungsfähigere Frameworks wie Apache Kafka oder Flink wären nur bei höheren Datenvolumen oder komplexeren Analysen sinnvoll, allerdings mit deutlich höherem Einrichtungsaufwand.

Anhang A: Python Code

```
import asyncio
import json
import time
from bleak import BleakClient, BleakScanner
import paho.mqtt.client as mqtt

# --- KONFIGURATION ---
ADDRESS = "98:07:2D:27:F1:86"
MQTT_BROKER = "IWILR4-11.CAMPUS.fh-ludwigshafen.de"
MQTT_TOPIC = "Sensor"

# UUIDs für den TI CC2650 SensorTag
SENSORS = {
    "humidity": {
        "service": "f000aa20-0451-4000-b000-000000000000",
        "data": "f000aa21-0451-4000-b000-000000000000",
        "config": "f000aa22-0451-4000-b000-000000000000",
        "period": "f000aa23-0451-4000-b000-000000000000"
    },
    "barometer": {
        "service": "f000aa40-0451-4000-b000-000000000000",
        "data": "f000aa41-0451-4000-b000-000000000000",
        "config": "f000aa42-0451-4000-b000-000000000000",
        "period": "f000aa44-0451-4000-b000-000000000000"
    },
    "optical_lux": {
        "service": "f000aa70-0451-4000-b000-000000000000",
        "data": "f000aa71-0451-4000-b000-000000000000",
        "config": "f000aa72-0451-4000-b000-000000000000",
        "period": "f000aa73-0451-4000-b000-000000000000"
    },
}
```

```
}
```

```
# --- MQTT SETUP ---
```

```
mqtt_client = mqtt.Client()
mqtt_client.connect(MQTT_BROKER, 1883, 60)
mqtt_client.loop_start()
```

```
def publish(type_name, value):
    payload = json.dumps({
        "device": "ti_cc2650",
        "type": type_name,
        "value": value,
        "timestamp": round(time.time(), 3),
    })
    mqtt_client.publish(MQTT_TOPIC, payload)
    print(f"Versendet: {type_name} -> {value}", flush=True)
```

```
def notification_handler(sender, data):
    """Verarbeitet eingehende Bluetooth-Daten und sendet sie per MQTT."""
    try:
        sender_uuid = sender.uuid if hasattr(sender, 'uuid') else str(sender)
        sensor_name = "Unknown"
        for name, uuids in SENSORS.items():
            if uuids["data"].lower() == sender_uuid.lower():
                sensor_name = name
                break

        if sensor_name == "humidity":
            raw_temp = int.from_bytes(data[0:2], byteorder='little')
            raw_hum = int.from_bytes(data[2:4], byteorder='little')
            publish("Temperature", round((raw_temp / 65536.0) * 165 - 40, 2))
            publish("Humidity", round((raw_hum / 65536.0) * 100, 2))
        elif sensor_name == "barometer":
```

```

raw_pres = int.from_bytes(data[3:6], byteorder='little')
publish("AirPressure", round(raw_pres / 100.0, 2))
elif sensor_name == "optical_lux":
    raw_lux = int.from_bytes(data, byteorder='little')
    m = raw_lux & 0x0FFF
    e = (raw_lux & 0xF000) >> 12
    publish("Illuminance", round(m * (0.01 * pow(2, e)), 2))

except Exception as e:
    print(f"Fehler in notification_handler: {e} | sender={sender} | data={data.hex()}", flush=True)

async def main(address):
    try:
        print("Suche nach dem SensorTag...")
        device = await BleakScanner.find_device_by_address(address, timeout=15.0)
        if device is None:
            print(f"Gerät {address} nicht gefunden.")
            return

        async with BleakClient(device) as client:
            print(f"Verbunden mit {address}")

            for name, uuids in SENSORS.items():
                # 1. Sensor einschalten (0x01 schreiben)
                await client.write_gatt_char(uuids["config"], b"\x01")
                # 2. Benachrichtigungen abonnieren
                await client.start_notify(uuids["data"], notification_handler)
                print(f"-> {name} aktiviert.")

            print("\nObservation läuft. Daten werden an MQTT gesendet...")
            while True:
                await asyncio.sleep(1)

```



```
except Exception as e:  
    print(f"Fehler: {e}")
```

```
# Programm starten
```

```
if __name__ == "__main__":  
    asyncio.run(main(ADDRESS))
```